

คู่มือทางเทคนิค • TECHNICAL HANDBOOK

คู่มือส่งมอบ พัฒนา และใช้งาน WashiOS FlightStack

Build • Flash • Diagnose • Create Payload Tasks

Branch: monorepo-migration

ฉบับปรับปรุง 24 กรกฎาคม 2026

ฉบับสำหรับ branch `monorepo-migration` | ปรับปรุง 24 กรกฎาคม 2026

เอกสารนี้เป็นคู่มือเริ่มต้นสำหรับผู้ที่ไม่เคยใช้ WashiOS มาก่อน ครอบคลุมตั้งแต่เตรียมเครื่อง, build, flash, ตรวจสอบอาการบนบอร์ด, ทดสอบ Payload UART/LaserCom ไปจนถึงการสร้าง FreeRTOS task สำหรับ payload ใหม่โดยไม่ทำให้ task-health และ watchdog ของระบบเสีย

เส้นทางอ่านที่แนะนำ:

- ต้องการทดลองทันที: อ่านหัวข้อ 3–14 แล้วทำตาม checklist
- ต้องการเพิ่ม payload task: อ่านหัวข้อ 15–16 และทำ checklist สำหรับนักพัฒนา
- ระบบไม่ boot, HardFault หรือ LED ไม่กระพริบ: ไปหัวข้อ Troubleshooting

1. ภาพรวม

WashiOS FlightStack คือชุดซอฟต์แวร์ฝังตัวสำหรับบอร์ด STM32 ที่สร้างอยู่บน FreeRTOS โดยเพิ่มส่วนประกอบที่จำเป็นต่อการทดลองระบบที่ต้องการความน่าเชื่อถือ เช่น bootloader ที่ตรวจสอบ firmware ก่อนเริ่มทำงาน, runtime core ที่มี task health และ watchdog, รวมถึง demo payload สำหรับทดสอบการสื่อสารระหว่างบอร์ดจริงสองตัว

ระบบนี้แบ่งเป็น 3 ส่วนหลัก:

- bootloader/ หรือ WashiBoot: firmware ตัวแรกที่รันหลัง reset ใช้ตรวจสอบ application ก่อน jump เข้า core
- core/ หรือ WashiOS-Core: application หลักที่รัน FreeRTOS task, watchdog, telemetry, laser communication และ payload link
- demo-payload/: firmware สำหรับบอร์ด payload responder อีกตัวหนึ่ง ใช้ตอบกลับบอร์ดหลักผ่าน UART

ลำดับการทำงานโดยรวม:

Power on / Reset

- > WashiBoot bootloader
- > ตรวจสอบ application image ใน Slot A หรือ Slot B
- > ถ้า image ถูกต้องจึง jump เข้า WashiOS-Core
- > WashiOS-Core เริ่ม FreeRTOS และรัน tasks
- > tasks ทำงาน เช่น heartbeat, watchdog, payload link, laser telemetry

2. โครงสร้าง Repository

หลังจัดเป็น monorepo แล้ว โครงสร้างหลักคือ:

```
WashiOS/  
  README.md  
  bootloader/  
  core/  
  demo-payload/  
  docs/  
  tools/
```

ความหมายของแต่ละโฟลเดอร์:

โฟลเดอร์	หน้าที่
bootloader/	โค้ด WashiBoot, linker script, unit test และ provisioning script
core/	WashiOS-Core, FreeRTOS, HAL abstraction, app tasks, protocol และ SITL tests

โฟลเดอร์	หน้าที่
demo-payload/	firmware สำหรับบอร์ด NUCLEO-G474RE ที่จำลอง payload responder
docs/	เอกสารคู่มือ, handoff สำหรับ research และ provisioning note
tools/	PowerShell scripts สำหรับ build, flash และ test

3. Hardware ที่ต้องใช้

อุปกรณ์	ใช้ทำอะไร
ST NUCLEO-G431RB	บอร์ดหลัก หรือ OBC ใช้รัน bootloader และ core
ST NUCLEO-G474RE	บอร์ด payload responder สำหรับ demo payload
สาย USB	ใช้จ่ายไฟและ flash ผ่าน ST-LINK
สาย jumper	ใช้ต่อ UART ระหว่างบอร์ด
laser module หรือ laser driver	ใช้สำหรับ laser communication demo
oscilloscope หรือ logic analyzer	ใช้ตรวจ PA6, UART และ pulse timing

ข้อควรระวัง: ห้ามต่อ laser diode เข้ากับ GPIO โดยตรง ให้ใช้ laser driver, transistor, MOSFET หรือ laser module ที่รับ logic 3.3 V ได้

4. Software ที่ต้องติดตั้ง

- Git

- Visual Studio Code (แนะนำ แต่ไม่บังคับ)
- PlatformIO Core หรือ PlatformIO Extension
- ST-LINK driver

ตรวจสอบ PlatformIO:

```
pio --version
```

ถ้า PowerShell หา pio ไม่เจอไม่จำเป็นต้องแก้ script: launcher ใน tools/ จะค้นหา pio/platformio จาก PATH และโฟลเดอร์ .platformio ของผู้ใช้โดยอัตโนมัติ หลังติดตั้ง PlatformIO แล้วให้ปิดและเปิด terminal ใหม่หนึ่งครั้ง

5. Memory Map

สำหรับ STM32G431RB:

ส่วน	Address	Size
Bootloader	0x08000000	16 KiB
Slot A	0x08004000	56 KiB
Slot B	0x08012000	56 KiB
RAM	0x20000000	32 KiB

เมื่อ reset บอร์ด MCU จะเริ่มที่ bootloader ก่อน จากนั้น bootloader จะตรวจ application ใน Slot A หรือ Slot B ถ้า image ถูกต้องจึง jump เข้า core

6. Pin Mapping สำคัญ

สำหรับ NUCLEO-G431RB:

Pin	หน้าที่
PA5	Heartbeat LED และ bootloader safe-loop beacon
PA6	Laser TX GPIO
PA2	USART2_TX debug console
PC4	USART1_TX payload UART
PC5	USART1_RX payload UART

สำหรับ laser communication:

```
PA6 -> input ของ laser driver  
GND -> ground ร่วมกับวงจร laser
```

7. วิธี Clone และเตรียม Project

clone repository:

```
git clone https://github.com/NatdanaiHS/WashiOS.git  
cd WashiOS  
git switch monorepo-migration
```

ตรวจว่าเห็นโครงสร้าง:

```
bootloader
core
demo-payload
docs
tools
README.md
```

8. หลักสำคัญก่อน Build และ Flash

ต้องจำกฎนี้เสมอ:

```
ต้อง build core ก่อน build bootloader
```

เหตุผลคือ bootloader จะอ่าน ELF ของ core, ตรวจสอบ vector table, คำนวณ CRC-32 แล้วฝังค่า CRC ลงใน build ของ bootloader ถ้าแก้ core แล้วไม่ rebuild bootloader ค่า CRC จะไม่ตรงและ bootloader อาจไม่ยอม boot application

ลำดับ build และ flash ที่ปลอดภัย:

1. build core environment ที่ต้องการ
2. build bootloader environment ที่ตรงกัน
3. upload core โดย **ยังไม่ reset**
4. upload bootloader ที่ฝัง CRC ของ core ตัวนั้น
5. reset เพียงครั้งเดียวเพื่อเริ่ม bootloader และ core ที่เป็นคนกัน

อย่า upload bootloader ก่อน core ด้วยคำสั่งแยกกัน เพราะบอร์ดอาจ reset แล้วพยายาม boot core เก่าที่ CRC ไม่ตรงในช่วงกลางกระบวนการ ใช้ `tools\flash_*.cmd` เป็นวิธีหลัก เพราะ script จัดลำดับนี้ให้อัตโนมัติ

launcher แบบ .cmd เหมาะสำหรับ Windows ทุกเครื่อง เพราะเรียก PowerShell ด้วย execution-policy แบบเฉพาะ process และค้นหา PlatformIO/OpenOCD ที่ติดตั้งในเครื่องให้เอง ไม่ต้องแก้ path ในไฟล์

9. ทดสอบ Native Tests

จาก repository root:

```
tools\test_native.ps1
```

หรือรันเอง:

```
cd bootloader  
pio test -e native
```

```
cd ..\core  
pio test -e native
```

10. ใช้งาน Laser Communication Demo

โมดนี้ใช้สำหรับทดสอบการส่งข้อมูลผ่าน laser ที่ถูกควบคุมด้วย GPIO

environment ที่ใช้:

```
nucleo_g431rb_lasercom
```

build และ flash แบบเร็ว:

```
cd <repo-root>  
tools\flash_lasercom.cmd
```

ถ้าต้องการ build อย่างเดียว:

```
tools\build_lasercom.ps1
```

คำสั่ง manual ต่อไปนี้ใช้สำหรับ build เท่านั้น:

```
cd core
pio run -e nucleo_g431rb_lasercom

cd ../bootloader
pio run -e nucleo_g431rb_lasercom
```

การ flash ให้ใช้ tools\flash_lasercom.cmd เพื่อรักษาลำดับ core-before-bootloader ที่ปลอดภัย

สิ่งที่ควรเห็น:

- PA5 หรือ LD2 กระพริบจาก heartbeat
- PA6 มี pulse สำหรับควบคุม laser
- ถ้าวัดด้วย oscilloscope จะเห็น sync pulse และ pulse สั้น/ยาวของข้อมูล

รูปแบบ pulse ปัจจุบัน:

Bit	High time	Low gap
0	2000 us	2000 us
1	4000 us	2000 us

ไฟล์ที่เกี่ยวข้อง:

```
core\include\comms\LaserPdmTx.hpp
core\src\app\LaserTelemetryTask.hpp
```

ข้อความ ASCII test ปัจจุบันอยู่ใน LaserTelemetryTask.hpp หากแก้ไขข้อความ ต้อง flash ใหม่ด้วย
tools\flash_lasercom.cmd

11. ใช้งาน Payload UART Demo

Payload UART demo ใช้บอร์ดสองตัว:

ฝั่ง	บอร์ด	โฟลเดอร์	หน้าที่
OBC	NUCLEO-G431RB	core/ + bootloader/	ส่ง poll และตรวจ response
Payload	NUCLEO-G474RE	demo-payload/	รับ poll และตอบ telemetry

environment ที่ใช้:

```
OBC core/bootloader = nucleo_g431rb_payload_demo
Payload responder = nucleo_g474re
```

flash ฝั่ง OBC G431:

```
cd <repo-root>
tools\flash_payload_demo.cmd
```

flash ฝั่ง payload responder G474:

```
cd <repo-root>
tools\flash_payload_responder.cmd
```

คำสั่ง manual สำหรับ payload responder:

```
cd demo-payload
pio run -e nucleo_g474re
pio run -e nucleo_g474re -t upload
```

ถ้าเสียบ ST-LINK สองบอร์ดพร้อมกัน ให้ระวัง flash ผิดบอร์ด ช่วงเริ่มต้นควรเสียบและ flash ที่ละบอร์ด

ลำดับสำหรับผู้เริ่มต้น:

1. ถอด G474 ออก เหลือ G431 แล้วรัน tools\flash_payload_demo.cmd
2. ถอด G431 ออก เสียบ G474 แล้วรัน tools\flash_payload_responder.cmd
3. ปิดไฟทั้งสองบอร์ด ต่อ UART และ GND ตามหัวข้อถัดไป
4. เสียบ USB ของทั้งสองบอร์ดแล้วเปิด serial monitor

12. Wiring ระหว่าง G431 และ G474

ต่อแบบ cross TX/RX:

G431 OBC	G474 Payload
D1 / PC4 / USART1_TX	D0 / PC5 / USART1_RX
D0 / PC5 / USART1_RX	D1 / PC4 / USART1_TX
GND	GND

ให้จ่ายไฟทั้งสองบอร์ดด้วย USB ของตัวเอง และอย่าต่อ 3V3 หรือ 5V ของสองบอร์ดเข้าหากัน

13. Serial Monitor

ฝั่ง OBC G431:

```
USART1 PC4/PC5 = คู่กับ payload
USART2 PA2      = debug console ผ่าน ST-LINK VCP
baud rate       = 115200
```

ฝั่ง payload G474:

```
USART1 PC4/PC5 = payload link
LPUART1 PA2/PA3 = debug console ผ่าน ST-LINK VCP
baud rate       = 115200
```

ดู COM port:

```
pio device list
```

เปิด monitor:

```
pio device monitor -p COM_OBC -b 115200
pio device monitor -p COM_PAYLOAD -b 115200
```

ให้แทน COM_OBC และ COM_PAYLOAD ด้วย COM port จริง

14. Fault Mode ของ Payload Responder

กด user button บน G474 เพื่อเปลี่ยนโหมด:

NORMAL -> SILENT -> BAD_CRC -> DELAYED -> NORMAL

Mode	ความหมาย
NORMAL	ตอบ telemetry ถูกต้อง
SILENT	รับ poll แต่ไม่ตอบ
BAD_CRC	ตอบกลับแต่ทำ CRC ให้ผิด
DELAYED	ตอบช้า 250 ms เกิน deadline 100 ms ของ OBC

LED บน G474 จะ toggle ทุกครั้งที่ได้รับ poll request ที่ถูกต้อง

สิ่งที่ควรเห็นใน console:

```
[PAYLOAD] READY board=NUCLEO-G474RE mode=NORMAL baud=115200
[PAYLOAD] LINK_ACTIVE
[PAYLOAD] MODE=SILENT
[PAYLOAD] MODE=BAD_CRC
[PAYLOAD] MODE=DELAYED
```

ฝั่ง OBC ควรเห็นสถานะเช่น:

```
[OBC] PAYLOAD_LINK_START baud=115200
[OBC] PAYLOAD_ONLINE ...
[OBC] PAYLOAD_TIMEOUT consecutive=...
[OBC] PAYLOAD_OFFLINE consecutive=...
[OBC] PAYLOAD_RECOVERED ...
```

payload offline ไม่ได้แปลว่า OBC firmware ตาย เพราะ PayloadLinkTask ยัง check-in กับ watchdog ได้อยู่ถ้า task ยังทำงานปกติ

15. Payload Protocol

OBC ส่ง PollRequest ทุกประมาณ 500 ms และ payload ต้องตอบ TelemetryResponse กลับมาด้วย sequence เดิม

frame มีขนาด 32 bytes:

Offset	Size	Field
0	1	sync0 = 0x57
1	1	sync1 = 0x50
2	1	version = 1
3	1	message type
4	4	sequence, little-endian
8	2	payload length, little-endian
10	2	flags, little-endian
12	16	payload
28	4	CRC-32 over bytes 0..27

message type:

Type	ค่า
PollRequest	0x01
TelemetryResponse	0x81

ถ้า CRC ผิด OBC จะ reject frame เป็น CRC error ถ้า sequence ผิดจะ reject เป็น sequence error
ถ้าไม่ตอบภายในเวลา OBC จะนับ timeout และถ้า timeout 3 ครั้งติดกันจะมองว่า payload offline

16. สร้าง Payload Task ใหม่ใน WashiOS-Core

หัวข้อนี้สอนการเพิ่มงานใหม่ที่รันอยู่บนบอร์ด OBC ภายใน core/ เช่น อ่าน sensor, ส่งอุปกรณ์ payload หรือส่งคำสั่งผ่าน UART ส่วน firmware ที่รันบนบอร์ด payload แยกต่างหากอยู่ใน demo-payload/ ทั้งสองอย่างไม่ใช่ไฟล์เดียวกัน:

ต้องการแก้พฤติกรรมที่ใด	จุดที่แก้
OBC จัดตารางงาน/สื่อสาร/เฝ้าระวัง payload	สร้าง task ใน core/src/app/
บอร์ด payload G474 ตอบคำสั่งหรืออ่าน sensor ของตัวเอง	แก้ firmware ใน demo-payload/
รูปแบบ packet ที่ทั้งสองฝั่งใช้ร่วมกัน	แก้ protocol และ test ทั้งสองฝั่งให้ตรงกัน

16.1 แบบจำลองของ task ในระบบ

task ของ WashiOS สืบทอดจาก `rtos_config::WashiTask<StackDepth>` ซึ่งเก็บ FreeRTOS control block และ stack แบบ static ไม่ใช่ heap ทุก task ที่ critical ต้อง:

1. มี TaskId ไม่ซ้ำกับ task ที่ทำงานใน environment เดียวกัน

2. ลงทะเบียนใน systemTaskHealth ก่อน scheduler เริ่ม
3. check-in ภายใน deadline อย่างสม่ำเสมอ
4. ถูกสร้างเป็น static object และเรียก Start() ใน main.cpp
5. ไม่ return ออกจาก Run() ในการทำงานปกติ

ค่าปัจจุบันใน core/src/main.cpp:

Task	TaskId	Health deadline
Heartbeat	1	1500 ms
Telemetry หรือ PayloadLink	2	800 ms
LaserTelemetry (เมื่อเปิดใช้)	3	2000 ms

TaskHealthRegistry<> ค่าเริ่มต้นรับได้ 8 task และ health summary mask แสดงเฉพาะ ID 0–31 การลงทะเบียน ID ซ้ำจะเขียนทับรายการเดิม จึงต้องเลือก ID ใหม่อย่างระมัดระวัง ตัวอย่างต่อไปใช้ ID 4

16.2 สร้างไฟล์ task

สร้าง core/src/app/FuturePayloadTask.hpp:

```
#pragma once

#include <cstdint>

#include "FreeRTOS.h"
#include "task.h"
#include "ITiming.hpp"
#include "TaskHealthReporter.hpp"
#include "WashiTask.hpp"

class FuturePayloadTask final : public rtos_config::WashiTask<384>
```

```

{
public:
    FuturePayloadTask(hal::ITiming& timingSource,
                     core::TaskHealthRegistry<>& registry,
                     core::TaskId taskId)
        : timing(timingSource)
    {
        healthReporter.configure(&registry, &timingSource, taskId);
    }

protected:
    void Run() override
    {
        for (;;)
        {
            (void)healthReporter.checkIn();
            samplePayload();
            vTaskDelay(pdMS_TO_TICKS(100));
        }
    }

private:
    hal::ITiming& timing;
    core::TaskHealthReporter<> healthReporter;
    uint32_t sampleCount = 0U;

    void samplePayload()
    {
        const uint64_t nowMs = timing.getSystemTick();
        (void)nowMs;
        ++sampleCount;
        // อ่าน sensor หรือส่ง payload แบบใช้เวลาจำกัดที่นี่
    }
};

```

หลักสำคัญของตัวอย่าง:

- 384 คือจำนวน StackType_t ไม่ใช่จำนวน byte ให้เริ่มจากค่าที่พอประมาณและวัด high-water mark ก่อนลด

- รับ HAL interface เช่น ITiming, IUart, IGPIO ผ่าน constructor/reference เพื่อให้ทดสอบด้วย mock ได้
- วงรอบทำงานทุก 100 ms ดังนั้น deadline ควรมากกว่า 100 ms หลายเท่าเพื่อเผื่อ scheduling และ I/O
- ห้ามใช้ new, malloc, container ที่โตได้ไม่จำกัด หรือ loop รอ UART โดยไม่มี timeout
- ถ้างานหนึ่งรอบอาจนาน ให้แบ่งเป็น state machine และจำกัดจำนวน byte/งานต่อรอบเหมือน PayloadLinkTask
- checkIn() หมายถึงตัว task ยังเดินอยู่ ไม่ได้หมายความว่าอุปกรณ์ payload online เสมอ สถานะ link/sensor ต้องรายงานแยกกัน

16.3 ผูก task เข้ากับ main.cpp

เพิ่ม include ใกล้เคียง task อื่น:

```
#include "app/FuturePayloadTask.hpp"
```

เพิ่ม ID และ deadline ใน anonymous namespace:

```
constexpr core::TaskId FuturePayloadTaskId = 4U;
constexpr uint32_t FuturePayloadDeadlineMs = 500U;
```

สร้าง object หลัง systemTaskHealth และ HAL object พร้อมใช้งานแล้ว:

```
static FuturePayloadTask futurePayloadTask(targetTiming,
                                           systemTaskHealth,
                                           FuturePayloadTaskId);
```

ลงทะเบียนก่อน start scheduler และตรวจผลลัพธ์ในงาน production:

```
const bool futurePayloadRegistered =
    systemTaskHealth.registerTask(FuturePayloadTaskId,
                                  FuturePayloadDeadlineMs,
                                  true,
                                  startupTimeMs);
```

ค่า `critical=true` หมายถึง watchdog จะไม่ถูก refresh เมื่อ task นี้หมด deadline และระบบจะเข้าสู่เส้นทางกู้คืน ถ้า payload ไม่จำเป็นต่อความปลอดภัยของ OBC ให้พิจารณา false โดยอ้างอิง requirement จริง ไม่ควรเลือกเพียงเพื่อให้ watchdog ไม่ reset

start task และรวมผลไว้ใน `tasksStarted`:

```
tasksStarted =
    futurePayloadTask.Start("FuturePayload", tskIDLE_PRIORITY + 2) &&
    tasksStarted;
```

อย่าละ `&& tasksStarted` เพราะ `handleBootTaskStartFailure()` ใช้ผลรวมนี้ตัดสินใจบันทึก fault และกู้คืนเมื่อสร้าง task ไม่สำเร็จ ชื่อ task ของ FreeRTOS ควรสั้นและสื่อความหมาย

16.4 เปิด task เฉพาะ environment ที่ต้องการ

ถ้า task ยังเป็นงานทดลอง ให้ครอบ include, object, registration และ `Start()` ด้วย macro เดียวกัน เช่น `WASHIOS_FUTURE_PAYLOAD` แล้วเพิ่ม environment:

```
[env:nucleo_g431rb_future_payload]
extends = env:nucleo_g431rb
build_flags =
    ${env:nucleo_g431rb.build_flags}
    -DWASHIOS_FUTURE_PAYLOAD
```

ต้องเพิ่ม environment ชื่อเดียวกันใน `bootloader/platformio.ini` โดยชี้ไป core environment ที่ตรงกัน เพื่อให้ bootloader provision CRC จาก ELF ที่ถูกต้อง วิธีที่ปลอดภัยที่สุดคือคัดรูปแบบ environment ของ `nucleo_g431rb_payload_demo` แล้วเปลี่ยนชื่อ/macro เท่านั้น

16.5 เพิ่ม UART หรือ GPIO

- ใช้ interface ใน `core/include/hal/` ภายใน task ไม่เรียก HAL global โดยตรง
- initialize peripheral ใน BSP/main ก่อนเริ่ม scheduler
- ส่ง reference ของ driver เข้าทาง constructor
- ทุก `readBuffer/writeBuffer` ต้องมี timeout สั้นและตรวจ return value
- จำกัด buffer แบบ compile-time เช่น `uint8_t frame[32] = {};`
- log ด้วย `FixedTextWriter<N>` แทน `sprintf` หรือ `std::string`
- ถ้ามี interrupt ให้ ISR ทำงานสั้นที่สุด ส่งข้อมูลต่อให้ task และตรวจ concurrency ให้ชัดเจน

16.6 ทดสอบก่อน flash

ขั้นต่ำที่ต้องทดสอบ:

1. logic ปกติสร้าง output ถูกต้อง
2. timeout/CRC/ข้อมูลผิดไม่ค้าง task
3. health check-in เกิดก่อน deadline
4. dependency เป็น null หรือ driver คืน failure แล้วระบบไม่ HardFault
5. buffer เต็มและ input ยาวผิดปกติไม่เขียนเกินขอบเขต
6. `Start()` และ registration failure ไปเส้นทาง safe recovery

ใช้ mock ที่มีอยู่ใน `core/test/mocks/` เช่น `MockTiming`, `MockUart` และเพิ่ม test ใน `core/test/test_sitl/test_main.cpp` จากนั้นรัน:

```
tools\test_native.ps1
cd core
pio run -e nucleo_g431rb_future_payload
```

เมื่อ build ผ่าน ให้เพิ่ม build_*.ps1, flash_*.ps1 และ .cmd โดยยึดรูปแบบ script ปัจจุบัน หรือส่ง environment ให้ script ที่รองรับ parameter ตัวอย่าง:

```
tools\flash_payload_demo.cmd -CoreEnv nucleo_g431rb_future_payload -BootloaderEnv
nucleo_g431rb_future_payload
```

16.7 Checklist ก่อนส่งมอบ task ใหม่

- TaskId ไม่ซ้ำ และจำนวน task ไม่เกิน registry capacity
- loop period สั้นกว่า health deadline พร้อม margin
- Run() มี loop ถาวรและมี vTaskDelay() หรือ block แบบมี timeout
- ไม่มี dynamic allocation และ buffer ทุกตัวมีขนาดจำกัด
- hardware access ผ่าน HAL interface/BSP
- register task ก่อน start และรวมผล Start() ใน tasksStarted
- environment ของ core และ bootloader ใช้ชื่อ/flag คู่กัน
- native tests ผ่าน, target build ผ่าน และ warning เป็นศูนย์
- flash ด้วย script ลำดับ core ก่อน bootloader
- ตรวจสอบ PA5 กระพริบทุกประมาณ 500 ms, console ไม่มี HardFault/reset loop และ payload ทำงานจริง

17. Bootloader ทำอะไร

bootloader/ หรือ WashiBoot เป็น firmware ตัวแรกหลัง reset หน้าที่หลักคือ:

1. ตรวจสอบ boot metadata
2. ตรวจสอบ retained fault log

3. ตรวจสอบ active slot
4. ตรวจสอบ vector table ของ application
5. ตรวจสอบ CRC-32 ของ application
6. ถ้าผ่านจึง jump เข้า application
7. ถ้า fail จะลอง fallback slot
8. ถ้าไม่มี slot ที่ใช้ได้ จะเข้า safe loop

safe loop คือสถานะที่ระบบไม่ jump เข้า application และใช้ PA5 toggle เป็น beacon เพื่อบอกว่าระบบอยู่ในสถานะปลอดภัย

18. Troubleshooting

อาการ	สาเหตุที่พบบ่อย	วิธีแก้
build bootloader แล้วหา core ELF ไม่เจอ	ยังไม่ได้ build core environment ที่ตรงกัน	build core ก่อน หรือใช้ script ใน tools/
upload core แล้วไม่รัน	CRC ไม่ตรง หรือยังไม่มี bootloader	ใช้ tools\flash_lasercom.cmd หรือ tools\flash_payload_demo.cmd
flash script หา pio ไม่เจอ	ยังไม่ได้ติดตั้ง PlatformIO หรือ terminal ยังใช้ environment เก่า	ติดตั้ง PlatformIO, เปิด terminal ใหม่ แล้วรัน .cmd อีกครั้ง
script ถูก PowerShell execution policy บล็อก	เปิด .ps1 โดยตรงบนเครื่องที่ policy เข้ม	ใช้ launcher .cmd ที่ชื่อเดียวกัน

อาการ	สาเหตุที่พบบ่อย	วิธีแก้
PA5 ไม่กระพริบหลัง flash	core ไม่เริ่ม, vector table ผิด, task start fail หรือ watchdog reset loop	flash ใหม่ด้วย .cmd, เปิด console, ตรวจสอบ VTOR ต้องชี้ Slot A 0x08004000 และดู retained fault log
PA5 กระพริบถี่/ช้าผิดปกติจากประมาณ 500 ms	กำลังอยู่ bootloader safe loop หรือ clock/tick ผิด	ตรวจสอบว่า core/bootloader เป็น env คู่กันและ CRC ตรง แล้วรีเซ็ต/reset/console
เพิ่ม task แล้ว reset ช้า	ไม่ check-in, deadline สั้นเกิน, stack ไม่พอ หรือ blocking I/O	เพิ่ม margin, จำกัด timeout, ตรวจสอบ stack high-water mark และ test failure path
เพิ่ม task แล้ว health ของ task อื่นผิด	ใช้ TaskId ซ้ำหรือ registry เต็ม	กำหนด ID ใหม่และเพิ่ม capacity อย่างมีเหตุผล
HardFault ทันทีหลัง jump เข้า core	vector table/VTOR หรือ image address ไม่ตรง linker slot	ใช้ linker/env ที่ถูกต้องและอย่าลบการตั้ง SCB->VTOR ก่อน HAL_Init()

อาการ	สาเหตุที่พบบ่อย	วิธีแก้
PA6 ไม่มี pulse	ใช้ env ผิด หรือ bootloader ไม่ jump เข้า core	ตรวจสอบ env, PA5 heartbeat, และ build/flash ตามลำดับ
laser ไม่ยิง แต่ PA6 มี pulse	hardware laser driver ต่อผิดหรือไม่รับ 3.3 V	ตรวจสอบ driver, GND, supply และ enable pin
payload ไม่ตอบ	ต่อ TX/RX ไม่ไขว้, ไม่ต่อ GND, COM/board ผิด	ตรวจสอบ wiring และ flash ที่ละบอร์ด
serial monitor ไม่เห็นข้อความ	เลือก COM ผิดหรือ baud ผิด	ใช้ pio device list และ baud 115200

19. Checklist ทดสอบ LaserCom

- clone repo แล้วเห็น bootloader/, core/, demo-payload/, docs/, tools/
- ติดตั้ง PlatformIO แล้ว
- ใช้บอร์ด NUCLEO-G431RB
- ต่อ PA6 ไปที่ input ของ laser driver ไม่ต่อ laser diode ตรง
- ต่อ GND ร่วม
- run tools\flash_lasercom.cmd สำเร็จ
- กด reset
- PA5 heartbeat กระพริบ
- PA6 มี pulse
- oscilloscope เห็น sync pulse และ pulse 2 ms / 4 ms

20. Checklist ทดสอบ Payload UART

- flash OBC G431 ด้วย tools\flash_payload_demo.cmd
- flash payload G474 ด้วย tools\flash_payload_responder.cmd
- ต่อ PC4/PC5 แบบ cross TX/RX
- ต่อ GND ร่วม
- ไม่ต่อ 3V3/5V ระหว่างบอร์ด
- เปิด serial monitor ที่ baud 115200
- เห็น payload READY
- เห็น OBC PAYLOAD_ONLINE
- กด button บน G474 แล้ว mode เปลี่ยน NORMAL/SILENT/BAD_CRC/DELAYED
- กลับมา NORMAL แล้ว OBC รายงาน recovery

21. ข้อจำกัดของระบบปัจจุบัน

- CRC-32 ใช้ตรวจ corruption แต่ไม่ใช่ security
- ยังไม่มี signed firmware verification
- ยังไม่มี anti-rollback
- boot metadata ยังเหมาะกับ development/prototype มากกว่า production
- LaserCom ยังเป็น transmitter demo ไม่ใช่ optical communication stack สมบูรณ์
- payload demo เป็น hardware-in-the-loop prototype ไม่ใช่ payload subsystem จริง
- ยังควรเพิ่ม fault-injection experiment และ measurement ก่อนใช้เป็น paper ที่แข็งแรง

22. สรุป

ถ้าต้องการทดสอบ laser communication:

```
cd <repo-root>
tools\flash_lasercom.cmd
```

ถ้าต้องการทดสอบ payload UART demo:

```
cd <repo-root>
tools\flash_payload_demo.cmd
tools\flash_payload_responder.cmd
```

สิ่งที่ต้องจำที่สุดมีสามข้อ:

1. build core ก่อน bootloader และ flash core แบบไม่ reset ก่อนลง bootloader ที่เป็นคู่กัน
2. task ใหม่ต้องมี TaskId ไม่ซ้ำ, ลงทะเบียน health, check-in หัน deadline และรวมผล Start()
3. ใช้ script ใน tools/ จาก repository root เพื่อให้ขั้นตอนเหมือนกันทุกเครื่อง